Министерство образования Республики Беларусь

Учреждение образования

«Брестский государственный технический университет»

Кафедра ИИТ

Лабораторная работа № 5

По дисциплине : «Модели решения задач в ИС»

По теме: «Бинарная классификация логических функций n переменных с использованием бинарной кросс-энтропии»

Выполнил:

Студент 3 курса

Группы ИИ-26

Рудь В.В.

Проверил:

Андренко К.В.

2026

Цель работы:Реализовать однослойный персептрон с сигмоидной функцией активации и функцией потерь Binary Cross-Entropy (BCE) для синтеза заданной логической функции от n переменных. Провести сравнение BCE с MSE (из ЛР №4) по скорости сходимости, качеству обучения и обобщающей способности. Исследовать влияние адаптивного шага обучения на BCE.

№ of variant	n	Task
1	5	OR

Задачи лабораторной работы:

1.Сгенерировать полную таблицу истинности для своей логической функции (2^n примеров).

2.Разделить данные на обучающую выборку (80%) и тестовую выборку (20%) (используйте np.random.seed(42) для воспроизводимости).

3.На основе кода из ЛР №4 (где уже есть сигмоида и адаптивный шаг) создать новую версию:

Заменить функцию потерь MSE на бинарную кросс-энтропию (BCE).

Реализовать правильный градиент для BCE + сигмоида:

4.Реализовать четыре конфигурации обучения (все в онлайн-режиме — один пример за обновление, как в предыдущих лабах):

А. MSE + фиксированный шаг (результаты можно взять из ЛР №4)

Б. MSE + адаптивный шаг (из ЛР №4)

В. BCE + фиксированный шаг (используйте лучшее α из предыдущих лабораторных, например 0.1 или 0.5)

Г. BCE + адаптивный шаг (по формуле из ЛР №2)

Критерий остановки для всех – суммарная ошибка Es ≤ Ee (возьмите Ee из ЛР №2 или №3, например 0.05 или 0.01).

5,Для каждой конфигурации:

Запускать обучение и сохранять значение суммарной ошибки после каждой эпохи.

Зафиксировать количество эпох до достижения критерия остановки.

После обучения посчитать accuracy на обучающей и тестовой выборках.

Проверить, сколько процентов из всей таблицы истинности (2^n примеров) сеть предсказывает правильно.

6,Построить один общий график зависимости суммарной ошибки Es от номера эпохи – все четыре кривые на одних осях (разными цветами/стилями).

7.Реализовать режим функционирования сети:

Пользователь вводит вектор из n значений (0 или 1).

Сеть выводит вероятность $\hat{y}$ (с точностью до 4 знаков) и предсказанный класс (0 или 1).

Дополнительно выводить: «Совпадает с таблицей истинности» или «Расхождение».

8.Написать вывод (обязательно ответить на следующие пункты):

Какой из методов сходится быстрее (MSE или BCE)? Насколько BCE ускоряет/замедляет обучение по сравнению с MSE?

Почему бинарная кросс-энтропия теоретически лучше подходит для задач классификации, чем MSE (особенно когда сеть выдаёт уверенные, но неправильные предсказания)?

Как изменилась обобщающая способность (accuracy на тесте и % восстановления полной таблицы) при переходе на BCE?

Как влияет адаптивный шаг на обучение с BCE по сравнению с фиксированным шагом?

Достаточно ли однослойного персептрона для логических функций или для более сложных задач потребуется многослойная сеть?

Код программы

```
import numpy as np

import matplotlib.pyplot as plt

from itertools import product

def main():

    n = 5

    task = "OR"


    EE = 0.01

    MAX_EPOCHS = 5000


    ALPHA_MSE_FIXED = 0.1

    ALPHA_BCE_FIXED = 0.1


    ALPHA0_MSE_ADAPT = 0.1

    ALPHA0_BCE_ADAPT = 0.1

    K_ADAPT = 0.01

    def logic_function(x):

        if task == "OR":

            return int(any(x))
```

```
    raise ValueError("Unknown task")



X = np.array(list(product([0, 1], repeat=n)))

y = np.array([logic_function(x) for x in X]).reshape(-1, 1)


print("=== ПОЛНАЯ ТАБЛИЦА ИСТИННОСТИ ===")

for x_vec, y_val in zip(X, y):

    print(f"{x_vec} -> {int(y_val[0])}")

np.random.seed(42)

indices = np.random.permutation(len(X))

train_size = int(0.8 * len(X))


train_idx = indices[:train_size]

test_idx = indices[train_size:]


X_train, y_train = X[train_idx], y[train_idx]

X_test, y_test = X[test_idx], y[test_idx]


print("\n=== РАЗДЕЛЕНИЕ ВЫБОРКИ ===")

print("Использован np.random.seed(42) для воспроизводимости.")

print(f"Всего примеров: {len(X)}")

print(f"Обучающая выборка (80%): {len(X_train)}")

print(f"Тестовая выборка (20%): {len(X_test)}")


def sigmoid(x):

    return 1 / (1 + np.exp(-x))

def mse_loss_and_grad(y_true, y_pred):

    # y_true, y_pred — скаляры
```

```
    error = y_pred - y_true
    loss = 0.5 * error ** 2
    dnet = error * y_pred * (1 - y_pred)  # (y_pred - y_true)*sigmoid'(net)
    return float(loss), float(dnet)


def bce_loss_and_grad(y_true, y_pred):
    # y_true, y_pred — скаляры
    eps = 1e-7
    y_pred_clipped = np.clip(y_pred, eps, 1 - eps)
    loss = -(y_true * np.log(y_pred_clipped) +
             (1 - y_true) * np.log(1 - y_pred_clipped))
    dnet = y_pred - y_true
    return float(loss), float(dnet)


def train_perceptron_online(
    X_train_, y_train_,
    X_test_, y_test_,
    loss_type="mse",
    alpha_init=0.1,
    adaptive=False,
    max_epochs=MAX_EPOCHS,
    ee=EE,
):
    m_train, n_features = X_train_.shape


    rng = np.random.default_rng(1)
    w = rng.uniform(-1, 1, size=(n_features, 1))
    b = float(rng.uniform(-1, 1))
```

```
if loss_type == "mse":
    loss_grad_fn = mse_loss_and_grad
elif loss_type == "bce":
    loss_grad_fn = bce_loss_and_grad
else:
    raise ValueError("loss_type must be 'mse' or 'bce'")


history_train = []
history_test = []


for epoch in range(max_epochs):
    if adaptive:
        alpha = alpha_init / (1 + K_ADAPT * epoch)
    else:
        alpha = alpha_init


    perm = np.random.permutation(m_train)
    X_train_shuffled = X_train_[perm]
    y_train_shuffled = y_train_[perm]


    epoch_loss_sum = 0.0


            for x_vec, y_true in zip(X_train_shuffled, y_train_shuffled):
        x_vec = x_vec.reshape(-1, 1)            # (n, 1)
        y_true = float(np.squeeze(y_true))       # скаляр 0 или 1


        net = (np.dot(x_vec.T, w) + b).item()     # скаляр
        y_pred = float(sigmoid(net))
```

```
            loss, dnet = loss_grad_fn(y_true, y_pred)
            epoch_loss_sum += loss

            w -= alpha * dnet * x_vec
            b -= alpha * dnet

        Es_train = epoch_loss_sum / m_train
        history_train.append(Es_train)

                test_loss_sum = 0.0
        for x_vec, y_true in zip(X_test_, y_test_):
            x_vec = x_vec.reshape(-1, 1)
            y_true = float(np.squeeze(y_true))

            net = (np.dot(x_vec.T, w) + b).item()     # скаляр
            y_pred = float(sigmoid(net))
            loss, _ = loss_grad_fn(y_true, y_pred)
            test_loss_sum += loss

        Es_test = test_loss_sum / len(X_test_)
        history_test.append(Es_test)

        if Es_train <= ee:
            break

    epochs_done = epoch + 1
    return w, b, np.array(history_train), np.array(history_test), epochs_done
results = {}
```

```
w_mse_fix, b_mse_fix, hist_mse_fix_tr, hist_mse_fix_te, ep_mse_fix = train_perceptron_online(
    X_train, y_train, X_test, y_test,
    loss_type="mse",
    alpha_init=ALPHA_MSE_FIXED,
    adaptive=False,
)

w_mse_ad, b_mse_ad, hist_mse_ad_tr, hist_mse_ad_te, ep_mse_ad = train_perceptron_online(
    X_train, y_train, X_test, y_test,
    loss_type="mse",
    alpha_init=ALPHA0_MSE_ADAPT,
    adaptive=True,
)
w_bce_fix, b_bce_fix, hist_bce_fix_tr, hist_bce_fix_te, ep_bce_fix = train_perceptron_online(
    X_train, y_train, X_test, y_test,
    loss_type="bce",
    alpha_init=ALPHA_BCE_FIXED,
    adaptive=False,
)

w_bce_ad, b_bce_ad, hist_bce_ad_tr, hist_bce_ad_te, ep_bce_ad = train_perceptron_online(
    X_train, y_train, X_test, y_test,
    loss_type="bce",
    alpha_init=ALPHA0_BCE_ADAPT,
    adaptive=True,
)

results["MSE_fixed"] = (w_mse_fix, b_mse_fix, hist_mse_fix_tr, hist_mse_fix_te, ep_mse_fix)
results["MSE_adapt"] = (w_mse_ad, b_mse_ad, hist_mse_ad_tr, hist_mse_ad_te, ep_mse_ad)
```

```
results["BCE_fixed"] = (w_bce_fix, b_bce_fix, hist_bce_fix_tr, hist_bce_fix_te, ep_bce_fix)
results["BCE_adapt"] = (w_bce_ad, b_bce_ad, hist_bce_ad_tr, hist_bce_ad_te, ep_bce_ad)



def predict_proba(X_, w_, b_):
    net = X_ @ w_ + b_
    return sigmoid(net)


def accuracy(X_, y_, w_, b_):
    probs = predict_proba(X_, w_, b_)
    y_pred = (probs >= 0.5).astype(int)
    return np.mean(y_pred == y_)


print("\n=== РЕЗУЛЬТАТЫ ОБУЧЕНИЯ (n = 5, OR) ===")
for name, (w_, b_, hist_tr, hist_te, epochs_done) in results.items():
    train_acc = accuracy(X_train, y_train, w_, b_)
    test_acc = accuracy(X_test, y_test, w_, b_)
    full_acc = accuracy(X, y, w_, b_)
    print(f"\n--- {name} ---")
    print(f"Эпох до останова: {epochs_done}")
    print(f"Accuracy train: {train_acc:.4f}")
    print(f"Accuracy test : {test_acc:.4f}")
    print(f"Accuracy full : {full_acc:.4f}")
plt.figure(figsize=(10, 6))
plt.plot(results["MSE_fixed"][2], label="MSE, фикс. шаг")
plt.plot(results["MSE_adapt"][2], label="MSE, адаптивный шаг")
plt.plot(results["BCE_fixed"][2], label="BCE, фикс. шаг")
plt.plot(results["BCE_adapt"][2], label="BCE, адаптивный шаг")
```

```
plt.xlabel("Эпоха")
plt.ylabel("Средняя ошибка Es на обучении")
plt.title("Сравнение MSE и BCE (онлайн-обучение, n = 5, OR)")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
w_best, b_best, _, _, _ = results["BCE_adapt"]


def predict_single(x_vec, w_, b_):
    x_vec = np.array(x_vec).reshape(1, -1)
    prob = float(predict_proba(x_vec, w_, b_)[0, 0])
    cls = int(prob >= 0.5)
    return prob, cls


print("\n=== РЕЖИМ ФУНКЦИОНИРОВАНИЯ СЕТИ (BCE + адаптивный шаг) ===")
print("Вводите 5 значений (0 или 1), разделённых пробелом. Для выхода введите 'q'.")


while True:
    s = input("x1 x2 x3 x4 x5 = ").strip()
    if s.lower() == "q":
        print("Выход из режима функционирования.")
        break


    try:
        parts = s.split()
        if len(parts) != n:
            print(f"Нужно ввести ровно {n} значений.")
            continue
```

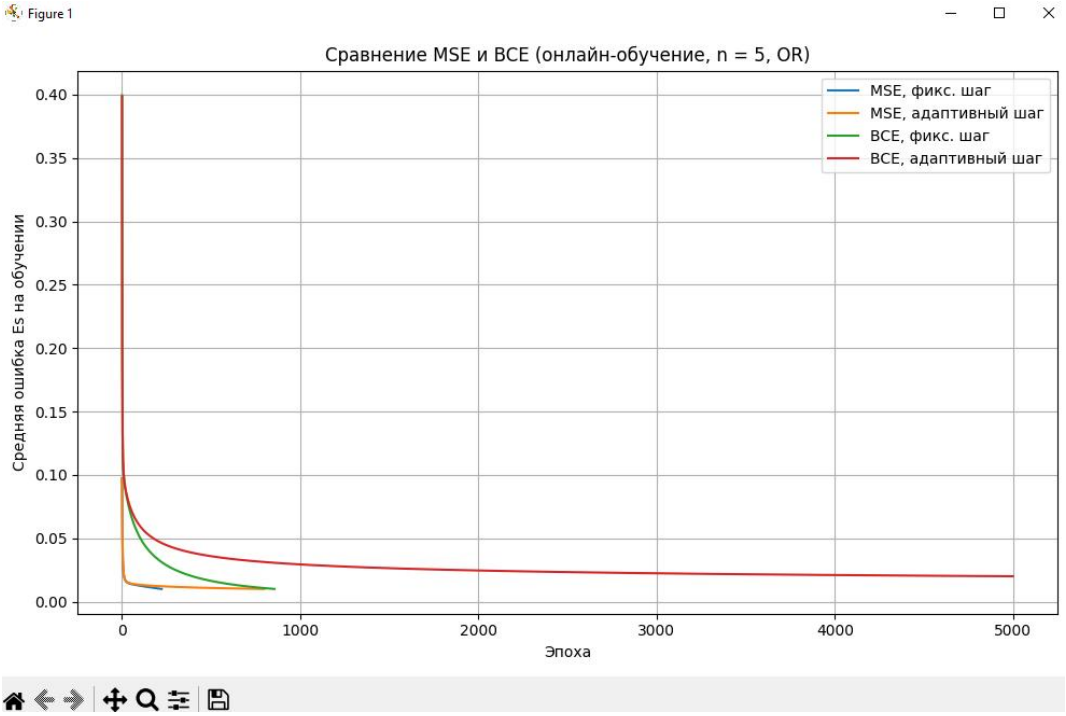
```
        x_input = [int(v) for v in parts]
        if any(v not in (0, 1) for v in x_input):
            print("Все значения должны быть 0 или 1.")
            continue

        prob, cls = predict_single(x_input, w_best, b_best)
        y_true = logic_function(x_input)

        print(f"Вход: {x_input}")
        print(f"Вероятность класса 1: {prob:.4f}")
        print(f"Предсказанный класс: {cls}")
        if cls == y_true:
            print("Совпадает с таблицей истинности.")
        else:
            print("Расхождение с таблицей истинности.")
    except Exception as e:
        print("Ошибка ввода, попробуйте ещё раз.", e)


if __name__ == "__main__":
    main()
```

Результат программы




```
=== ПОЛНАЯ ТАБЛИЦА ИСТИННОСТИ ===
[0 0 0 0 0] -> 0
[0 0 0 0 1] -> 1
[0 0 0 1 0] -> 1
[0 0 0 1 1] -> 1
[0 0 1 0 0] -> 1
[0 0 1 0 1] -> 1
[0 0 1 1 0] -> 1
[0 0 1 1 1] -> 1
[0 1 0 0 0] -> 1
[0 1 0 0 1] -> 1
[0 1 0 1 0] -> 1
[0 1 0 1 1] -> 1
[0 1 1 0 0] -> 1
[0 1 1 0 1] -> 1
[0 1 1 1 0] -> 1
[0 1 1 1 1] -> 1
[1 0 0 0 0] -> 1
[1 0 0 0 1] -> 1
[1 0 0 1 0] -> 1
[1 0 0 1 1] -> 1
[1 0 1 0 0] -> 1
[1 0 1 0 1] -> 1
[1 0 1 1 0] -> 1
[1 0 1 1 1] -> 1
[1 1 0 0 0] -> 1
[1 1 0 0 1] -> 1
[1 1 0 1 0] -> 1
[1 1 0 1 1] -> 1
[1 1 1 0 0] -> 1
[1 1 1 0 1] -> 1
[1 1 1 1 0] -> 1
[1 1 1 1 1] -> 1
```

```
=== РАЗДЕЛЕНИЕ ВЫБОРКИ ===
Использован np.random.seed(42) для воспроизводимости.
Всего примеров: 32
Обучающая выборка (80%): 25
Тестовая выборка (20%): 7

=== РЕЗУЛЬТАТЫ ОБУЧЕНИЯ (n = 5, OR) ===

--- MSE_fixed ---
Эпох до останова: 223
Accuracy train: 0.9600
Accuracy test : 1.0000
Accuracy full : 0.9688

--- MSE_adapt ---
Эпох до останова: 799
Accuracy train: 0.9600
Accuracy test : 1.0000
Accuracy full : 0.9688

--- BCE_fixed ---
Эпох до останова: 856
Accuracy train: 1.0000
Accuracy test : 1.0000
Accuracy full : 1.0000

--- BCE_adapt ---
Эпох до останова: 5000
Accuracy train: 1.0000
Accuracy test : 1.0000
Accuracy full : 1.0000
```

```
=== РЕЖИМ ФУНКЦИОНИРОВАНИЯ СЕТИ (BCE + адаптивный шаг) ===
Вводите 5 значений (0 или 1), разделённых пробелом. Для выхода введите 'q'.
x1 x2 x3 x4 x5 = 1 1 1 1 1
Вход: [1, 1, 1, 1, 1]
Вероятность класса 1: 1.0000
Предсказанный класс: 1
Совпадает с таблицей истинности.
x1 x2 x3 x4 x5 = 0 0 0 0 0
Вход: [0, 0, 0, 0, 0]
Вероятность класса 1: 0.2430
Предсказанный класс: 0
Совпадает с таблицей истинности.
x1 x2 x3 x4 x5 = 1 0 0 0 1
Вход: [1, 0, 0, 0, 1]
Вероятность класса 1: 0.9994
Предсказанный класс: 1
Совпадает с таблицей истинности.
x1 x2 x3 x4 x5 = 2 2 1 1 0
Все значения должны быть 0 или 1.
x1 x2 x3 x4 x5 =
```

Задание №8

В ходе лабораторной работы был реализован однослойный персептрон с сигмоидной функцией активации для бинарной классификации логической функции OR при n = 5. Были сформированы все $2^n = 32$ комбинации входов (полная таблица истинности), после чего данные разделены на обучающую (80%) и тестовую (20%) выборки с использованием np.random.seed(42) для воспроизводимости результатов. Обучение выполнено в онлайн-режиме (одно обновление весов на один пример) для четырёх конфигураций: MSE + фиксированный шаг, MSE + адаптивный шаг, BCE + фиксированный шаг, BCE + адаптивный шаг.

По результатам эксперимента установлено, что по критерию останова Es ≤ Ee (при Ee = 0,01) быстрее сошлись конфигурации на MSE: MSE с фиксированным шагом — 223 эпохи, MSE с адаптивным шагом — 799 эпох. Для BCE с фиксированным шагом потребовалось 856 эпох, а BCE с адаптивным шагом достигла лимита 5000 эпох, не выполнив условие останова по Es в заданном диапазоне эпох. Следовательно, в данной конкретной настройке гиперпараметров BCE не ускорила обучение относительно MSE, а по числу эпох оказалась медленнее.

Теоретически бинарная кросс-энтропия лучше подходит для задач классификации, чем MSE, так как согласована с вероятностной постановкой бинарной задачи (Bernoulli-модель) и сильнее штрафует уверенные ошибочные предсказания. Если сеть выдаёт значение вероятности, близкое к 0 или 1, но неверного класса, BCE формирует более информативный обучающий сигнал, тогда как MSE в ряде случаев даёт менее эффективную коррекцию весов. Поэтому BCE обычно предпочтительнее именно для задач классификации.

При этом качество обобщения в эксперименте при переходе на BCE улучшилось: для BCE получено Accuracy_train = 1,00000, Accuracy_test = 1,00000, то есть 100% восстановление полной таблицы истинности. Для MSE получено Accuracy_train = 0,96000, Accuracy_test = 1,00000, Accuracy_full = 0,9688 (96,88% по полной таблице). Это показывает, что, несмотря на более медленную сходимость по выбранному критерию ошибки, BCE обеспечила лучшее итоговое качество классификации.

Влияние адаптивного шага для BCE в данном запуске оказалось отрицательным по скорости достижения критерия Es ≤ Ee: вариант с адаптивным шагом работал дольше, чем с фиксированным. Это связано с выбранными параметрами адаптации шага (начальное значение и скорость убывания), при которых шаг мог уменьшаться слишком быстро. Следовательно, адаптивный шаг требует отдельного подбора параметров; сам по себе он не гарантирует ускорения обучения.

Вывод: Однослойный персептрон достаточен для линейно разделимых логических функций (например, OR, AND), что подтверждается полученными результатами. Для более сложных нелинейно разделимых функций (например, XOR) возможностей однослойной модели недостаточно, и требуется многослойная нейронная сеть со скрытыми слоями.